

# Efficient Attentions for Long Document Summarization

Luyang Huang<sup>1</sup> Shuyang Cao<sup>1</sup> Nikolaus Parulian<sup>2</sup> Heng Ji<sup>2</sup> Lu Wang<sup>1</sup>

<sup>1</sup>Computer Science and Engineering, University of Michigan, Ann Arbor, MI

<sup>2</sup>Department of Computer Science, University of Illinois at Urbana-Champaign, IL

<sup>1</sup>{lyhuang, caoshuy, wangluxy}@umich.edu

<sup>2</sup>{nnp2, hengji}@illinois.edu

## Abstract

The quadratic computational and memory complexities of large Transformers have limited their scalability for long document summarization. In this paper, we propose **HEPOS**, a novel *efficient encoder-decoder attention with head-wise positional strides* to effectively pinpoint salient information from the source. We further conduct a systematic study of existing efficient self-attentions. Combined with HEPOS, we are able to process ten times more tokens than existing models that use full attentions. For evaluation, we present a new dataset, **GOVREPORT**, with significantly longer documents and summaries. Results show that our models produce significantly higher ROUGE scores than competitive comparisons, including new state-of-the-art results on PubMed. Human evaluation also shows that our models generate more informative summaries with fewer unfaithful errors.

## 1 Introduction

Long documents, such as scientific papers and government reports, often discuss substantial issues at length, and thus are time-consuming to read, let alone to comprehend. Generating abstractive summaries can help readers quickly grasp the main topics, yet prior work has mostly focused on short texts (containing hundreds of words), e.g., news articles (Gehrmann et al., 2018; Liu and Lapata, 2019; Zhang et al., 2019).

*Model training efficiency and summary quality* present a pair of challenges for long document summarization. State-of-the-art systems (Lewis et al., 2020; Zhang et al., 2019) are built upon Transformer (Vaswani et al., 2017), which uses attentions to compute pairwise relations between tokens. Such framework has quadratic time and memory complexities, and is too costly for long documents<sup>1</sup>. Solutions have been proposed to reduce

the calculation of *encoder self-attentions* (Wang et al., 2020c; Zaheer et al., 2020) by selectively attending to neighboring tokens (Beltagy et al., 2020; Child et al., 2019) or relevant words (Kitaev et al., 2020; Tay et al., 2020a). Yet, these methods do not apply to *encoder-decoder attentions* in summarization models since they collaborate and dynamically pinpoint salient content in the source as the summary is decoded. Truncation is commonly used to circumvent the issue. However, training on curtailed content further aggravates “hallucination” in existing abstractive models (Maynez et al., 2020).

We argue that summarizing long documents (e.g., with thousands of words or more) requires efficient handling of both types of attentions. To this end, we propose an efficient encoder-decoder attention with **head-wise positional strides (HEPOS)**, where the attention heads follow a strided pattern and have varying starting positions. HEPOS reduces computational and memory costs while (1) maintaining the power of emphasizing important tokens, and (2) preserving the global context per head. HEPOS successfully doubles the processed input sequence size, when combined with any encoder. To the best of our knowledge, we are the first to study efficient encoder-decoder attentions and provide a systematic comparison of diverse encoder attentions for the task of summarization.<sup>2</sup>

For evaluation, we collect a **new large-scale dataset, GOVREPORT**, consisting of about 19.5k U.S. government reports with expert-written abstractive summaries.<sup>3</sup> GOVREPORT has two important features: (1) It contains significantly longer documents (9.4k words) and summaries (553 words) than existing datasets, such as PubMed and arXiv (Cohan et al., 2018) (see Table 2); (2) Salient

tokens with a batch size of 1, 70GB of memory is needed for encoder attentions, and 8GB for encoder-decoder attentions.

<sup>2</sup>Our code is released at <https://github.com/luyang-huang96/LongDocSum>.

<sup>3</sup>GOVREPORT can be downloaded from <https://gov-report-data.github.io>.

<sup>1</sup>For instance, to fine-tune BART on documents of 10K

content is spread throughout the documents, as opposed to cases where summary-worthy words are more heavily concentrated in specific parts of the document. These properties make GOVREPORT an important benchmark for producing long document summaries with multiple paragraphs.

We conduct experiments on GOVREPORT and scientific papers in PubMed and arXiv. First, when summarizing documents of the same length, HEPOS *attention yields significantly better ROUGE scores* than a non-trivial comparison that projects attentions into low-rank space (Wang et al., 2020c). Second, when trained on the same GPU, HEPOS attention, combined with sparse encoder attentions, is able to read more than 10K words and obtains significantly higher ROUGE scores on GOVREPORT and new state-of-the-art results on PubMed, compared with full encoder-decoder attention models which can process at most 5K input words. Human judges further rate the summaries generated by our models to be *more informative and faithful*.

We further propose a **new evaluation metric for faithfulness**, inspired by APES (Eyal et al., 2019), a fill-in-the-blank QA metric for summary evaluation. With questions generated from references, our metric, APES<sub>src</sub>, compares QA answers by reading the source and the system summary. It is shown to be better correlated with human judgment than the original metric and an entailment-based scorer (Kryscinski et al., 2020).

The rest of the paper is organized as follows. We describe efficient encoder attentions in prior work in § 2, and formulate our proposed encoder-decoder attention in § 3. The GOVREPORT data is presented in § 4. We then share details on evaluation metrics (§ 5) and experimental results (§ 6). Additional related work is listed in § 7, with conclusion in § 8.

## 2 Prior Work on Efficient Encoder Attentions

Transformer models are built upon multi-head attentions in multiple layers. The attention is calculated as  $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}})\mathbf{V}$ , where  $\mathbf{Q}$ ,  $\mathbf{K}$ , and  $\mathbf{V}$  are query, key, and value matrices, each consisting of  $n$  vectors for a document with  $n$  tokens, thus the quadratic memory footprint.

Here, we present an overview of representative methods for efficient encoder self-attentions (henceforth “encoder attentions”) that can be built upon large pre-trained seq2seq models, e.g., BART (Lewis et al., 2020). We follow the naming

| Model                             | Complexity              | # New Para.      |
|-----------------------------------|-------------------------|------------------|
| <b>Full</b>                       | $\mathcal{O}(n^2)$      | —                |
| <b>Encoder Self-attentions</b>    |                         |                  |
| <i>I. Fixed Patterns</i>          |                         |                  |
| Sliding Window (2020)             | $\mathcal{O}(nw)$       | 0                |
| Adaptive Span (2019)              | $\mathcal{O}(n\hat{w})$ | $\mathcal{O}(1)$ |
| Global Tokens (2020)              | $\mathcal{O}(2ng)$      | 0                |
| Stride (2019)                     | $\mathcal{O}(n^2/s)$    | 0                |
| Random (2020)                     | $\mathcal{O}(nr)$       | 0                |
| <i>II. Low-rank</i>               |                         |                  |
| Linformer (2020c)                 | $\mathcal{O}(nk)$       | $\mathcal{O}(n)$ |
| <i>III. Learnable Patterns</i>    |                         |                  |
| LSH (2020)                        | $\mathcal{O}(\ln b_l)$  | 0                |
| Sinkhorn (2020a)                  | $\mathcal{O}(2nb_s)$    | 0                |
| <b>Encoder-decoder Attentions</b> |                         |                  |
| Hepos (ours)                      | $\mathcal{O}(mn/s_h)$   | 0                |
| Linformer                         | $\mathcal{O}(mk)$       | $\mathcal{O}(n)$ |

Table 1: Summary of efficient Transformer attentions on *memory complexity* and *newly learned parameters* compared with full attentions at each layer.  $m$  and  $n$  are lengths of the input and the output. See § 2 and § 3 for model-specific hyperparameters.

convention of Tay et al. (2020b), and summarize their *memory complexities* and numbers of *newly learned parameters* in Table 1.

### 2.1 Fixed Patterns

Fixed patterns are used to limit the scope of attentions. In our experiments, in addition to window-based attentions, we also combine them with global tokens, stride patterns, or random attentions.

**Sliding window attentions** (Beltagy et al., 2020) aim to capture the local context, which is critical for language understanding (Liu\* et al., 2018; Child et al., 2019). Concretely, each query token attends to  $w/2$  neighboring tokens on both left and right, yielding a memory complexity of  $\mathcal{O}(nw)$ .

**Adaptive span** is proposed by Sukhbaatar et al. (2019) to learn attention windows at different layers. This is implemented by learning a masking function for each head independently. In practice, the adaptive span attention has a complexity of  $\mathcal{O}(n\hat{w})$ , where  $\hat{w}$  is the maximum values of predicted spans for all heads. Besides, it introduces  $\mathcal{O}(1)$  new parameters for learning spans.

**Global tokens** (Beltagy et al., 2020) are often added to sliding windows to let pre-selected tokens attend to the full sequence, to build global representations. Importantly, global attention operations are symmetric, i.e., a global token is also attendable to all tokens in the sequence. We select the first  $g$  tokens as global tokens, as leading sentences are

often important for summarization. Memory complexity is  $\mathcal{O}(2ng)$  due to the symmetric attentions.

**Stride patterns** are proposed by Child et al. (2019) to capture long term interactions, where each query attends to every  $s$ -th token, with  $s$  as the stride size. It thus has a complexity of  $\mathcal{O}(n^2/s)$ .

**Random attention** is motivated by the fact that randomly constructed graphs with  $\tilde{\Theta}(n)$  edges can approximate the complete graphs spectrally (Zaheer et al., 2020). Zaheer et al. (2020) propose to allow each query to attend to  $r$  random keys, resulting in a complexity of  $\mathcal{O}(nr)$ . For efficient implementations, input tokens are first segmented into blocks. Tokens in the same block attend to tokens in another randomly selected block.

## 2.2 Low-rank Methods

Wang et al. (2020c) show that self-attention matrices are low-rank. They propose **Linformer** that linearly projects key and value matrices into a low-dimensional space, e.g., from  $n$  to  $k$ , to achieve a  $\mathcal{O}(nk)$  complexity. It also introduces  $\mathcal{O}(n)$  new parameters for projection matrix learning.

## 2.3 Learnable Patterns

Recently, learnable sparse attentions are proposed to better capture both local and global contexts than attentions based on fixed patterns.

**Locality-sensitive hashing (LSH) attentions** use a random-projection hashing function to hash similar queries and keys into the same buckets in  $l$  rounds (Kitaev et al., 2020). Attentions are then computed among tokens within each bucket. For bucket size  $b_l$ , the complexity of LSH attention is  $\mathcal{O}(\ln b_l)$ .

**Sinkhorn attentions** first segment a sequence into blocks, which are then arranged by a learned Sinkhorn sorting network (Tay et al., 2020a). Given the new permutation, each query attends to  $b_s$  tokens within the same block to maintain the local context and another  $b_s$  tokens in a neighboring block to capture global interactions. Its complexity is  $\mathcal{O}(2nb_s)$ .

## 2.4 Other Attentions

We also describe several notable methods that are not suitable for our experiments and excluded from this study: **Recurrence** over input segments are tailored for an autoregressive decoder only (Dai et al., 2019); **memory** methods use a separate memory module to attend to full sequences (Lee et al.,

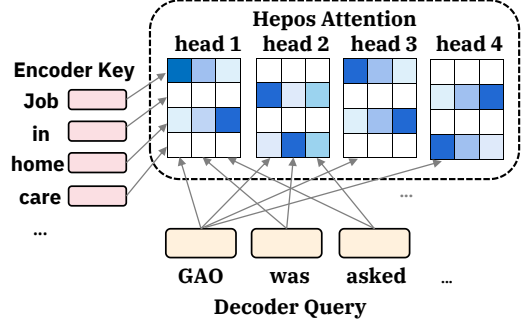


Figure 1: A toy example of our HEPOS attention, with a stride of 2 and four attention heads. Dark colors indicate that heads 1 and 3 attend to the first and third tokens (“Job” and “home”) in the input, heads 2 and 4 look at the second and fourth words (“in” and “care”).

2019), which share a similar theoretical foundation as global tokens; and **kernel** methods over attentions require training models from scratch (Choromanski et al., 2020; Katharopoulos et al., 2020).

## 3 Encoder-decoder Attention with Head-wise Positional Strides (Hepos)

The efficient design of encoder-decoder attentions with head-wise positional strides (HEPOS) allows models to consume longer sequences. Concretely, our design is motivated by two observations: (1) Attention heads are redundant (Voita et al., 2019). (2) Any individual head rarely attends to several tokens in a row (Clark et al., 2019). Therefore, as illustrated in Fig. 1, HEPOS uses separate encoder-decoder heads on the same layer to cover different subsets of source tokens at fixed intervals. Each head starts at a different position, and all heads collectively attend to the full sequence.

Given a stride size of  $s_h$ , for the  $h$ -th head, its attention value between decoder query  $\mathbf{q}_j$  (at step  $j$ ) and encoder key vector  $\mathbf{k}_i$  (for the  $i$ -th input token) can be formulated as:

$$a_{ji}^h = \begin{cases} \text{softmax}(\mathbf{q}_j \mathbf{k}_i), & \text{if } (i - h) \bmod s_h = 0 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

In HEPOS attention, each query token attends to  $n/s_h$  tokens per head, yielding a memory complexity of  $\mathcal{O}(mn/s_h)$ , where  $m$  is the output length.

For **comparison**, Linformer (§ 2.2) can be straightforwardly adapted for encoder-decoder attentions by using decoder queries for attention calculation instead. We do not adapt pattern-based attentions (§ 2.1 and § 2.3), since they rely on local token grouping which makes it difficult to pinpoint salient content.